

Name: Aarya Balwadkar

23070122005

Foundations of DevOps and Version Control

Git Setup & Basic Workflow:

Step 1: Create a New Repository on GitHub

1. Log in to your GitHub account.
 2. Click the + icon in the top-right corner and select "New repository".
 3. Enter a **Repository name** (e.g., my-git-practice).
 4. Optionally, add a **Description**.
 5. Choose between **Public** (visible to anyone) or **Private** (you control access). For this exercise, either is fine.
 6. **Important:** Do **NOT** initialize the repository with a README, .gitignore, or license file *yet*. We will create the README locally. Leave these options unchecked.
 7. Click the "Create repository" button.
 8. On the next page, you'll see instructions and a URL (HTTPS or SSH). **Copy the HTTPS URL**. It will look something like `https://github.com/your-username/my-git-practice.git`.
-

Step 2: Clone the Repository to Your Local Machine

1. Open your terminal or command prompt.
2. Navigate to the directory where you want to store your project locally using the `cd` (change directory) command. For example: `cd Documents/Projects`.
3. Use the `git clone` command followed by the URL you copied in Step 1:
Bash
`git clone <paste_the_repository_url_here>`
Replace `<paste_the_repository_url_here>` with the actual HTTPS URL.
Example:
Bash
`git clone https://github.com/your-username/my-git-practice.git`
or Bash
`git clone https://gitlab.com/your-username/my-git-practice.git`
4. Git will download the repository (which is currently empty except for hidden Git configuration files) into a new folder named after your repository (e.g., my-git-practice).
5. Navigate into your new project directory:

```
Bash
cd my-git-practice
```

Step 3: Create a File and Add it to Staging

1. Inside the `my-git-practice` directory, create a simple text file named `README.md`. You can do this using:
 - A text editor (like VS Code, Notepad, Sublime Text, Vim, Nano): Create the file, type some text (e.g., `# My Git Practice Project`), and save it as `README.md`.
 - The command line:
 - (Linux/macOS/Git Bash): `echo "# My Git Practice Project" > README.md`
 - (Windows Command Prompt): `echo # My Git Practice Project > README.md`
 - (Windows PowerShell): `echo "# My Git Practice Project" | Out-File -Encoding UTF8 README.md`
 2. Now, check the status of your repository:

```
Bash
git status
```

You should see `README.md` listed under "Untracked files". This means Git sees the file, but isn't tracking changes to it yet.
 3. Tell Git you want to track this new file and include it in the next snapshot (commit) by adding it to the staging area:

```
Bash
git add README.md
```

Alternatively, you could use `git add .` to stage all new or modified files in the current directory.
 4. Check the status again:

```
Bash
git status
```

You should now see `README.md` listed under "Changes to be committed".
-

Step 4: Commit the Staged Changes

1. Now that the file is staged, commit it to your local repository's history. Use the `git commit` command with the `-m` flag to provide a descriptive message:

```
Bash
git commit -m "Add initial README.md file"
```

 - The `-m` flag allows you to write the commit message inline. If you omit it (`git commit`), Git will usually open a text editor for you to write a longer message.
 - Good commit messages are crucial! They explain *why* a change was made.
2. Check the status one more time:

```
Bash
git status
```

It should now say something like "nothing to commit, working tree clean" and mention that your local branch is ahead of `origin/main` (or `origin/master`).

Step 5: Push the Commit to the Remote Repository

1. Your commit currently only exists on your local machine. To share it with the remote repository (GitHub/GitLab), use the `git push` command:

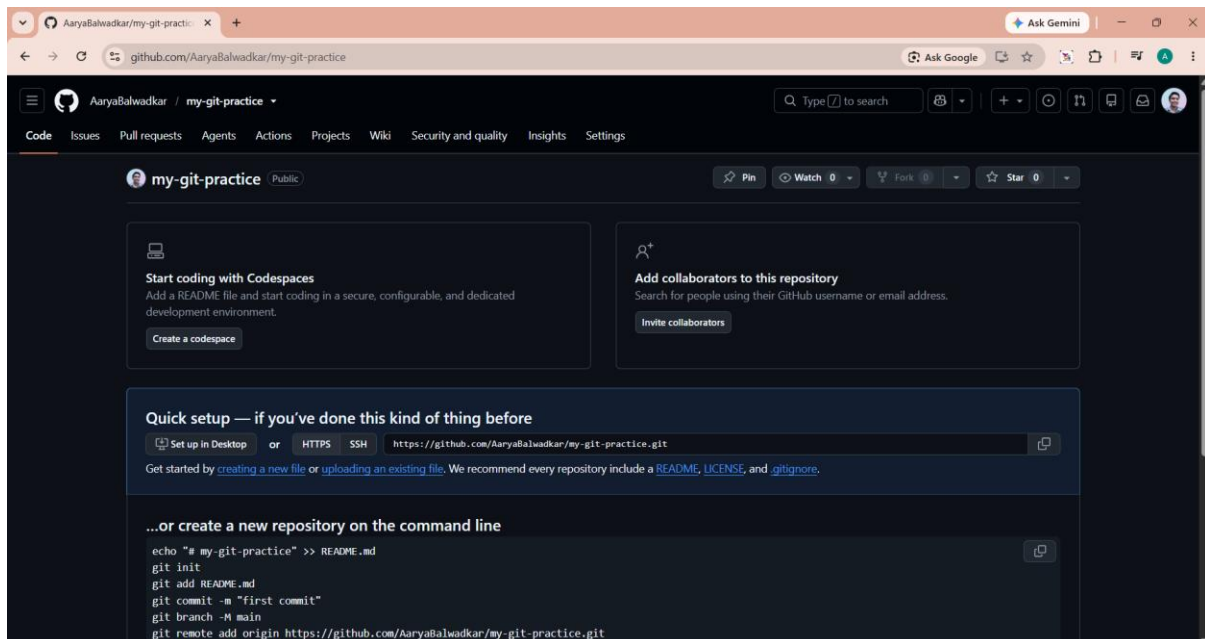
Bash

```
git push origin main
```

- o `origin`: This is the default short name Git gives to the remote repository you cloned from.
 - o `main`: This is the name of the default branch in most modern Git repositories (historically, it was often `master`). If you cloned an older repository or your setup uses `master`, you might need to use `git push origin master` instead. Your `git status` output in the previous step might give you a hint about the branch name.
2. You might be prompted to enter your GitHub/GitLab username and password (or a Personal Access Token, which is more secure and often required now instead of passwords).
 3. Once the push is successful, your local commit(s) have been uploaded to the remote repository.
-

Verification:

- Go back to your repository page on GitHub or GitLab in your web browser.
- Refresh the page.
- You should now see your `README.md` file listed, along with the commit message you used ("Add initial README.md file").



```
MINGW64/SEM_7
Aarya@LAPTOP-BBTIK5FL MINGW64 /f/SEM_7
$ git remote add origin https://github.com/AaryaBalwadar/my-git-practice.git
git branch -M main
git push -u origin main
fatal: not a git repository (or any of the parent directories): .git
fatal: not a git repository (or any of the parent directories): .git
fatal: not a git repository (or any of the parent directories): .git

Aarya@LAPTOP-BBTIK5FL MINGW64 /f/SEM_7
$ git clone git@github-outlook:AC

Aarya@LAPTOP-BBTIK5FL MINGW64 /f/SEM_7
$ git clone git@github-outlook:AaryaBalwadar/my-git-practice.git
Cloning into 'my-git-practice'...
warning: You appear to have cloned an empty repository.

Aarya@LAPTOP-BBTIK5FL MINGW64 /f/SEM_7
$ |
```

```
Final Help  ← → my-git-practice
Welcome  ⓘ README.md X
  ⓘ README.md > abc # My Git Practice Project
1 # My Git Practice Project

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\SEM_7\my-git-practice> git status

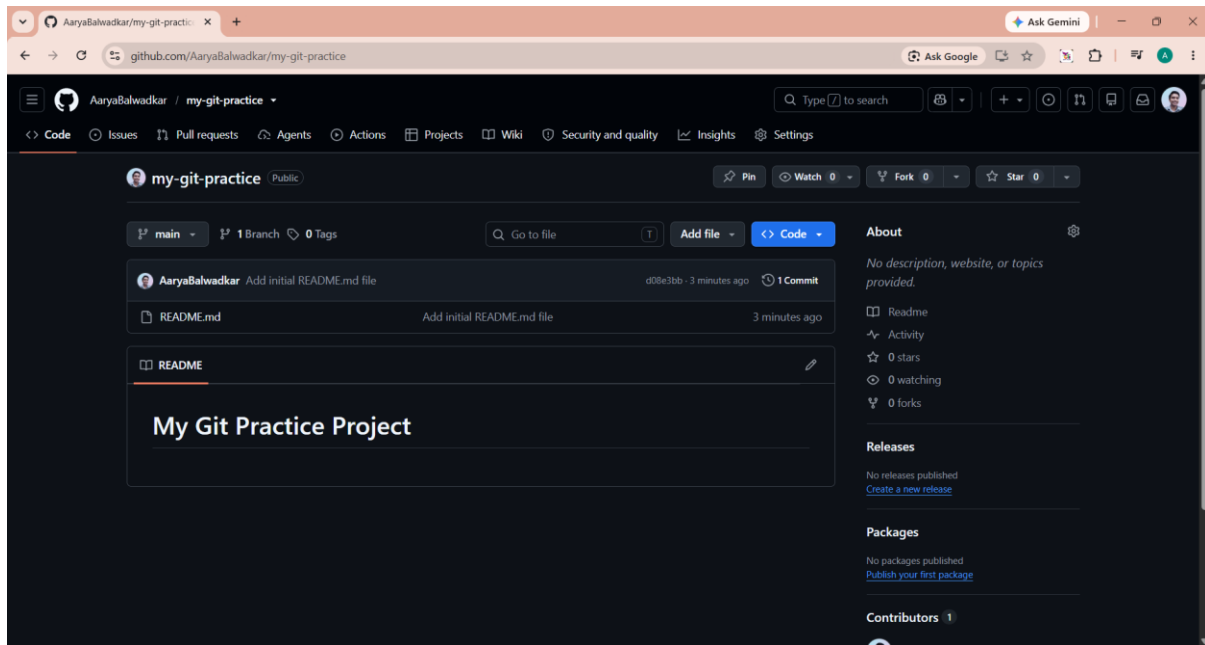
nothing added to commit but untracked files present (use "git add" to track)
• PS F:\SEM_7\my-git-practice> git add .
• PS F:\SEM_7\my-git-practice> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md

• PS F:\SEM_7\my-git-practice> git commit -m "Add initial README.md file"
[main (root-commit) d08e3bb] Add initial README.md file
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
• PS F:\SEM_7\my-git-practice> git status
On branch main
Your branch is based on 'origin/main', but the upstream is gone.
  (use "git branch --unset-upstream" to fixup)

nothing to commit, working tree clean
• PS F:\SEM_7\my-git-practice> git branch --unset-upstream
• PS F:\SEM_7\my-git-practice> git status
On branch main
nothing to commit, working tree clean
• PS F:\SEM_7\my-git-practice> git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 259 bytes | 259.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github-outlook:AaryaBalwadar/my-git-practice.git
 * [new branch]    main -> main
• PS F:\SEM_7\my-git-practice> |
```



Git Branching & Merging:

Step 1: Verify Current State

1. Make sure you are in your project's root directory in your terminal.

Bash

```
# Example: cd ~/Documents/Projects/my-git-practice
```

2. Check your current branch and status to ensure everything is clean and you are on the main branch:

Bash

```
git status
```

- It should report you are on branch main (or master).
- It should say "nothing to commit, working tree clean".
- It should indicate your branch is up to date with origin/main (or origin/master).

Step 2: Create and Switch to a New Branch

1. Create a new branch called feature-update. The convention is often to use prefixes like

feature/, bugfix/, hotfix/, but feature-update is fine for this exercise.

- You can create and switch branches in two steps:

Bash

```
# git branch feature-update # Creates the branch
```

```
# git checkout feature-update # Switches to the branch
```

- *Or, more commonly, do both in one step using checkout -b:*

Bash

```
git checkout -b feature-update
```

2. Verify you are now on the new branch:

Bash

```
git branch
```

- This command lists all local branches. The current branch will have an asterisk (*) next to it (e.g., * feature-update).
- git status will also show On branch feature-update.

Step 3: Modify File, Add, and Commit on the Feature Branch

1. Open the README.md file in a text editor.
2. Add a new line of text, for example: "This update adds details about a new feature." Save the file.

3. Check the status to see the modification:

Bash

```
git status
```

- You'll see README.md listed under "Changes not staged for commit".

4. Stage the changes using git add:

Bash

```
git add README.md
```

5. Commit the staged changes *to the feature-update branch*:

Bash

```
git commit -m "Update README on feature branch"
```

6. Check the status again:

Bash

```
git status
```

- It should now report "nothing to commit, working tree clean" for the `feature-update` branch.

Step 4: Switch Back to the Main Branch

1. Switch back to your main development branch. This is typically `main` or `master`.

Bash

```
git checkout main
```

(If your main branch is named `master`, use `git checkout master`)

2. **Observe:** Open `README.md` again or view its content using `cat README.md` (Linux/macOS) or `type README.md` (Windows). You will see that the line you added in Step 3 is **gone**. This is because you are now back on the `main` branch, which does not yet have the commit you made on `feature-update`.

Step 5: Merge the Feature Branch into the Main Branch

1. Ensure you are still on the receiving branch (`main` or `master`). `git status` or `git branch` can confirm this.
2. Execute the merge command, specifying the branch you want to merge *into* the current branch (`main`):
Bash

```
git merge feature-update
```
3. Git will perform the merge. Since you didn't make any other changes on `main` after creating `feature-update`, this will likely be a "fast-forward" merge. Git simply moves the `main` branch pointer forward to point to the same commit as `feature-update`. The output might look like:
4. Updating <hash1>..<hash2>
5. Fast-forward
6. README.md | 1 +
7. 1 file changed, 1 insertion(+)
(If there had been divergent changes on both branches, Git would attempt a true merge, potentially creating a new "merge commit").
8. **Observe:** Check the content of `README.md` again. The line you added on the `feature-update` branch should now be present in the file while you are on the `main` branch.

Step 6: Push the Updated Main Branch

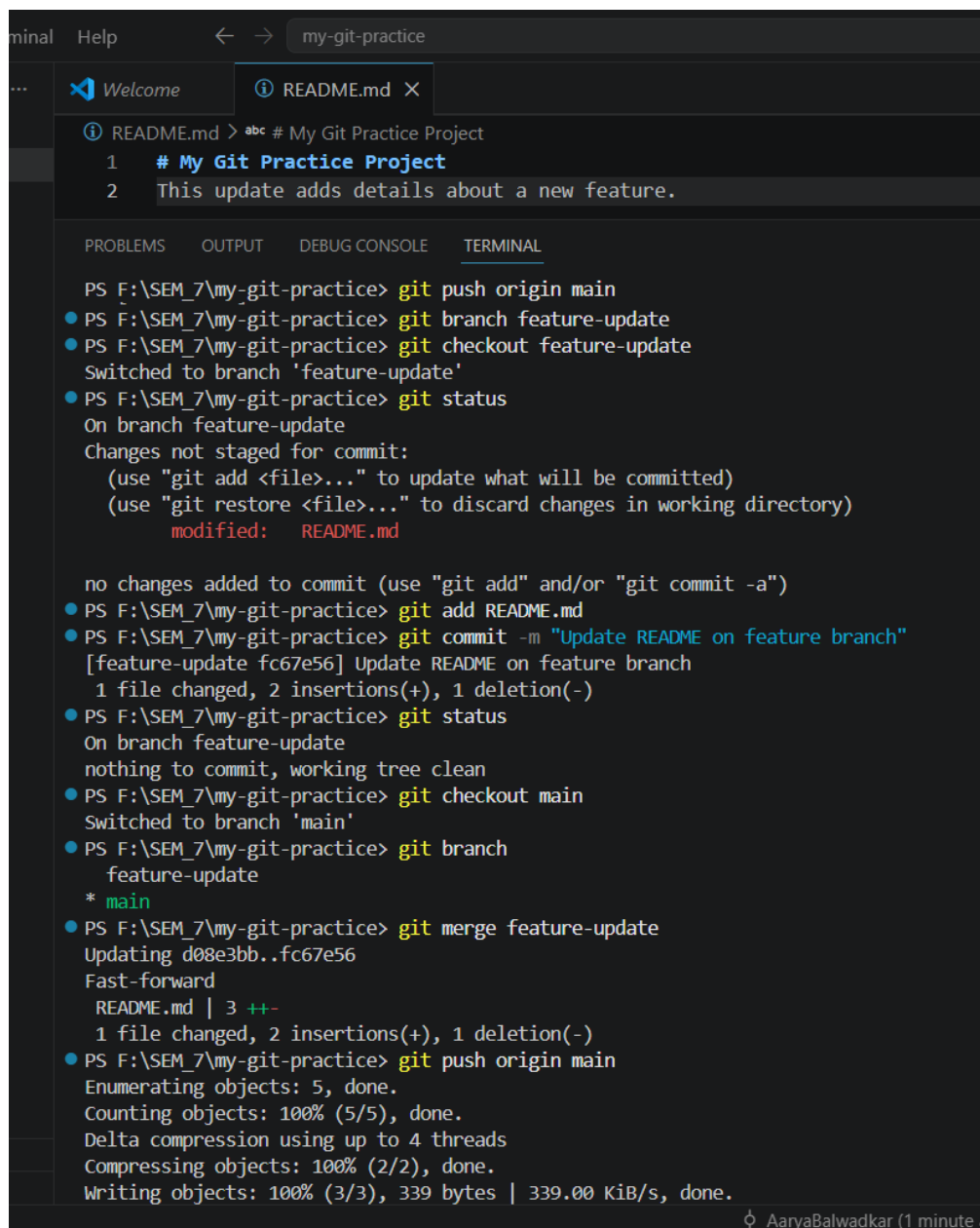
1. The merge has updated your *local* `main` branch. Now, you need to update the *remote* `main` branch on GitHub/GitLab.
2. Push the changes from your local `main` branch to `origin` (the remote):
Bash

```
git push origin main
```

(Again, use `master` if that's your main branch name: `git push origin master`)

Verification:

- Go to your repository page on GitHub or GitLab in your web browser.
- Make sure you are viewing the `main` (or `master`) branch.
- Refresh the page.
- The `README.md` file should show the content including the line added on the feature branch.
- Check the commit history online. You should see both your initial commit and the "Update README on feature branch" commit (or a merge commit depending on the exact history).



The screenshot shows a VS Code editor window with a file named `README.md` open. The file content is as follows:

```
1 # My Git Practice Project
2 This update adds details about a new feature.
```

Below the editor, the `TERMINAL` tab is active, displaying a series of Git commands and their outputs in a PowerShell prompt:

```
PS F:\SEM_7\my-git-practice> git push origin main
PS F:\SEM_7\my-git-practice> git branch feature-update
PS F:\SEM_7\my-git-practice> git checkout feature-update
Switched to branch 'feature-update'
PS F:\SEM_7\my-git-practice> git status
On branch feature-update
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
PS F:\SEM_7\my-git-practice> git add README.md
PS F:\SEM_7\my-git-practice> git commit -m "Update README on feature branch"
[feature-update fc67e56] Update README on feature branch
 1 file changed, 2 insertions(+), 1 deletion(-)
PS F:\SEM_7\my-git-practice> git status
On branch feature-update
nothing to commit, working tree clean
PS F:\SEM_7\my-git-practice> git checkout main
Switched to branch 'main'
PS F:\SEM_7\my-git-practice> git branch
  feature-update
* main
PS F:\SEM_7\my-git-practice> git merge feature-update
Updating d08e3bb..fc67e56
Fast-forward
 README.md | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
PS F:\SEM_7\my-git-practice> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 339 bytes | 339.00 KiB/s, done.
```

The terminal output indicates that the `feature-update` branch was created, the changes to `README.md` were staged and committed, the `main` branch was checked out, and the `feature-update` branch was merged into `main`. Finally, the merged state was pushed to the origin.


```
terminal Help  ← → my-git-practice

Welcome  README.md X

README.md > abc # My Git Practice Project
1  # My Git Practice Project
2  This update adds details about a new feature.

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

• PS F:\SEM_7\my-git-practice> git status
On branch feature-update
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
• PS F:\SEM_7\my-git-practice> git add README.md
• PS F:\SEM_7\my-git-practice> git commit -m "Update README on feature branch"
[feature-update fc67e56] Update README on feature branch
 1 file changed, 2 insertions(+), 1 deletion(-)
• PS F:\SEM_7\my-git-practice> git status
On branch feature-update
nothing to commit, working tree clean
• PS F:\SEM_7\my-git-practice> git checkout main
Switched to branch 'main'
• PS F:\SEM_7\my-git-practice> git branch
  feature-update
* main
• PS F:\SEM_7\my-git-practice> git merge feature-update
Updating d08e3bb..fc67e56
Fast-forward
 README.md | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
• PS F:\SEM_7\my-git-practice> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 339 bytes | 339.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github-outlook:AaryaBalwadkar/my-git-practice.git
 d08e3bb..fc67e56  main -> main
• PS F:\SEM_7\my-git-practice>
```

Simulating Git Collaboration & Conflict Resolution:

Step 1: Make a Change on the main Branch

1. Ensure you are in your project's root directory.
 2. Switch to your main branch (main or master) and make sure it's clean:
Bash

```
git  
checko  
ut main  
git  
status
```

If not clean, commit or stash changes first.
If collaborating, run 'git pull origin main' to get latest updates.
 3. Open README.md in a text editor.
 4. Modify the **first line** of the file. For example, if it currently says # My Git Practice Project, change it to:
Markdown

```
# My Git Practice Project (Main Update)
```

(Make sure to remember roughly what the original line was for the next step).
 5. Save the file.
 6. Stage and commit this change on the main branch:
Bash

```
git add README.md  
git commit -m "Update README first line on main"
```
-

Step 2: Make a Conflicting Change on a Feature Branch

1. Create and switch to a new branch. Let's call it conflicting-feature. (If you still have feature-update from the last exercise and deleted it, you can reuse that name. Using a new name avoids confusion).
Bash

```
git checkout -b conflicting-feature
```

Note: This branch is created based on the commit before the one you just made on main.
2. Open README.md again in the text editor. Because you branched off *before* the commit in Step 1, the first line should still be the original version (e.g., # My Git Practice Project).
3. Modify the **same first line** but make a *different* change than you did on main. For example:
Markdown

```
# Awesome Git Practice Project (Feature Update)
```
4. Save the file.
5. Stage and commit this change on the conflicting-feature branch:
Bash

```
git add README.md
```

```
git commit -m "Update README first line on feature branch"
```

Step 3: Attempt the Merge and Trigger the Conflict

1. Switch back to the branch you want to merge *into*, which is main:
Bash

```
git checkout main
```
 2. Attempt to merge the conflicting-feature branch into main:
Bash

```
git merge conflicting-feature
```
 3. **Observe the Output:** Git will try to merge automatically, fail, and report the conflict. The output will look something like this:
 4. Auto-merging README.md
 5. CONFLICT (content): Merge conflict in README.md
 6. Automatic merge failed; fix conflicts and then commit the result.
 7. Check the status:
Bash

```
git status
```

 - Git will explicitly tell you that you have "unmerged paths".
 - README.md will be listed under this section.
 - It will also give you hints: (fix conflicts and run "git commit") and (use "git merge --abort" to abort the merge).
-

Step 4: Identify and Resolve the Conflict

1. Open the conflicting file (README.md) in your text editor.
2. Look for the conflict markers that Git inserted. They clearly delimit the conflicting sections from both branches:
Markdown

```
<<<<<<< HEAD
# My Git Practice Project (Main Update)
=====
# Awesome Git Practice Project (Feature Update)
>>>>>>> conflicting-feature
(Any other content in the file will be outside these markers)
```

 - <<<<<<< HEAD: Marks the beginning of the content from your current branch (main because HEAD points to main).
 - =====: Separates the conflicting content from the two branches.
 - >>>>>>> conflicting-feature: Marks the end of the content from the branch you are trying to merge (conflicting-feature).
3. **Resolve the conflict:** You need to manually edit this section to look exactly how you want the final version to be. This involves:
 - **Deciding** which change to keep, or combining them, or writing something entirely new.

- **Deleting** all the conflict markers (<<<<<< HEAD, =====, >>>>>> conflicting-feature).
 - **Example Resolution:** Let's decide to combine the ideas into a new first line. Edit the file so the entire content looks like this (removing the markers and choosing the final version):
Markdown
My Awesome Git Practice Project (Resolved
Conflict) (Any other content in the file)
4. Save the README.md file after making your edits.
-

Step 5: Stage and Commit the Resolution

1. After editing the file and saving your resolution, you need to tell Git that you've resolved the conflict in that file by staging it:
Bash

```
git add README.md
```
 2. Check the status again:
Bash

```
git status
```

 - It should now say All conflicts fixed but you are still merging.
 - README.md will be listed under "Changes to be committed".
 3. Complete the merge by making a commit. Git usually prepares a default merge commit message for you. You can simply run:
Bash

```
git commit
```

 - This will likely open your default text editor (like Vim, Nano) with a pre-filled message, such as Merge branch 'conflicting-feature'.
 - You can leave this message as is, or modify it if you wish.
 - Save and close the editor (e.g., in Vim, type :wq and press Enter; in Nano, press Ctrl+X, then Y, then Enter).
 - *Alternatively, you could provide a message directly:*

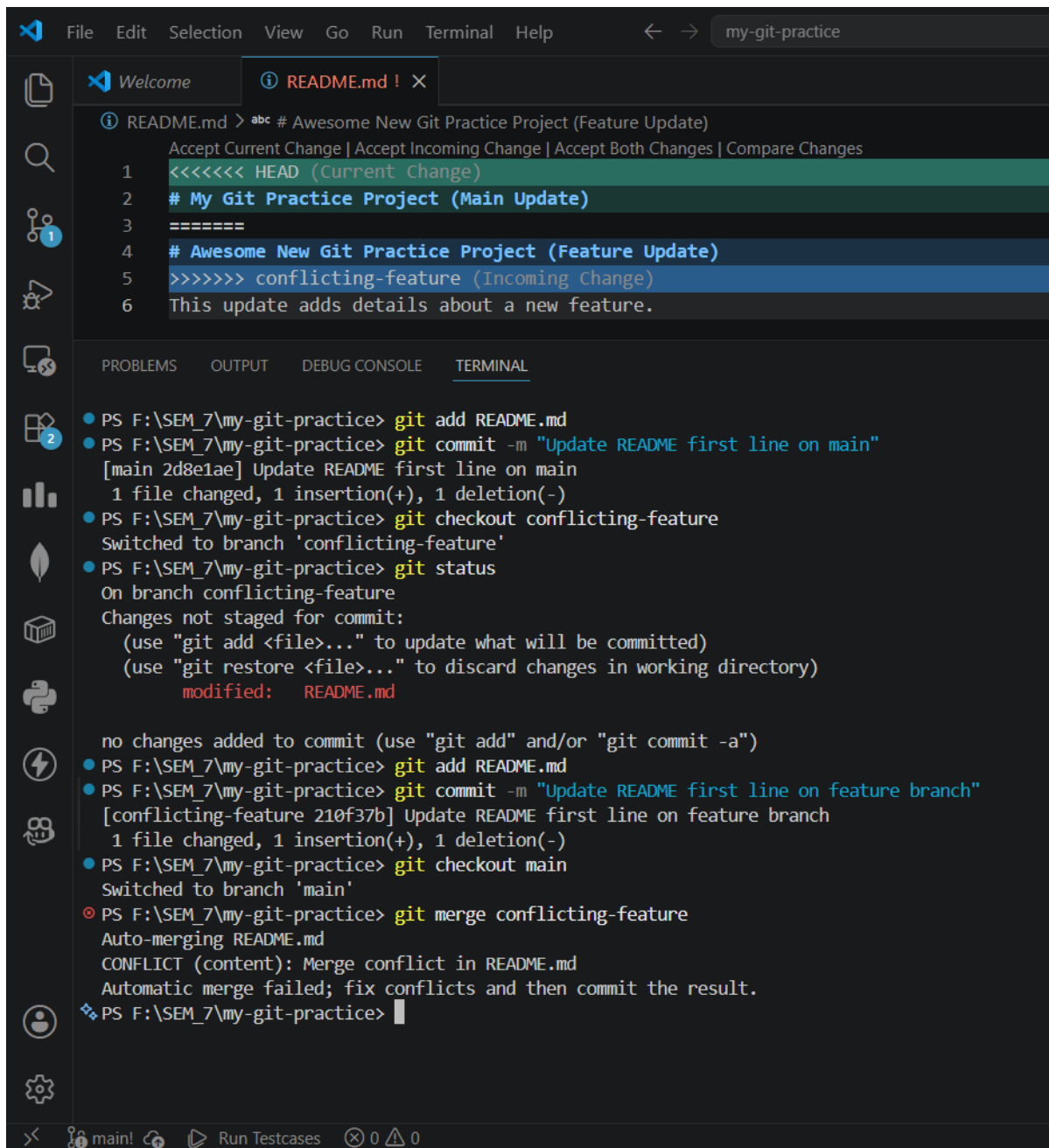
```
git commit -m "Merge branch 'conflicting-feature', resolving README conflict"
```
-

Verification:

- Check the content of README.md locally; it should match your resolved version.
- Examine

```
git log --oneline --graph
```

; you should see the merge commit connecting the two lines of development.
- If pushed, check the main branch on GitHub/GitLab. The README.md file should be updated, and the commit history should reflect the merge.

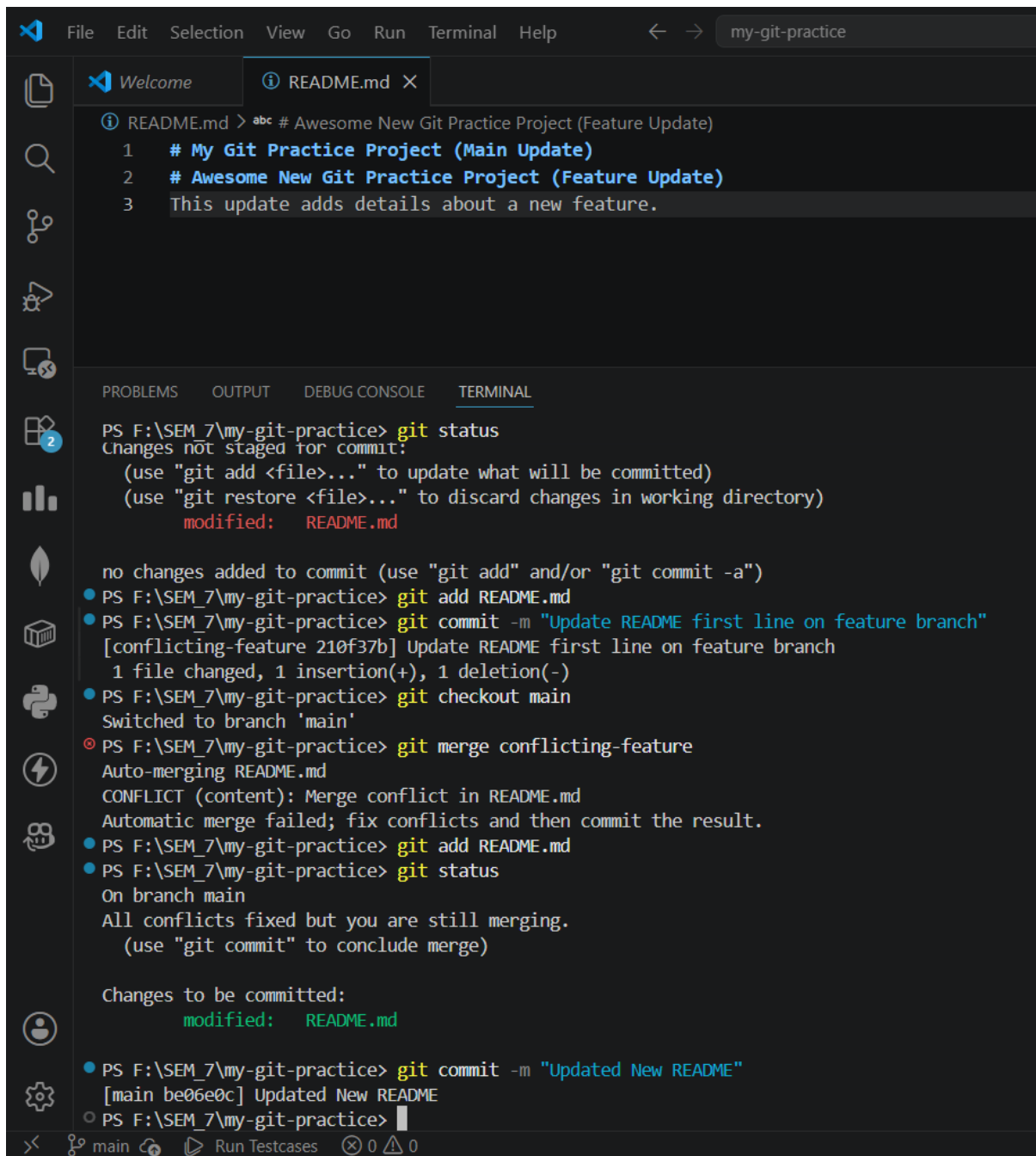


The image shows a Visual Studio Code editor window titled "my-git-practice". The editor is open to a file named "README.md", which contains a merge conflict. The conflict is between the "HEAD (Current Change)" and the "conflicting-feature (Incoming Change)". The current change has a line "# My Git Practice Project (Main Update)" and the incoming change has a line "# Awesome New Git Practice Project (Feature Update)". The conflict is resolved by keeping the current change.

The terminal window shows the following commands and output:

```
PS F:\SEM_7\my-git-practice> git add README.md
PS F:\SEM_7\my-git-practice> git commit -m "Update README first line on main"
[main 2d8e1ae] Update README first line on main
1 file changed, 1 insertion(+), 1 deletion(-)
PS F:\SEM_7\my-git-practice> git checkout conflicting-feature
Switched to branch 'conflicting-feature'
PS F:\SEM_7\my-git-practice> git status
On branch conflicting-feature
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
PS F:\SEM_7\my-git-practice> git add README.md
PS F:\SEM_7\my-git-practice> git commit -m "Update README first line on feature branch"
[conflicting-feature 210f37b] Update README first line on feature branch
1 file changed, 1 insertion(+), 1 deletion(-)
PS F:\SEM_7\my-git-practice> git checkout main
Switched to branch 'main'
PS F:\SEM_7\my-git-practice> git merge conflicting-feature
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS F:\SEM_7\my-git-practice>
```



The screenshot shows the Visual Studio Code interface with a dark theme. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The breadcrumb at the top right indicates the file path: my-git-practice. The editor has two tabs: 'Welcome' and 'README.md'. The 'README.md' tab is active, showing the following content:

```
1 # My Git Practice Project (Main Update)
2 # Awesome New Git Practice Project (Feature Update)
3 This update adds details about a new feature.
```

Below the editor is a panel with four tabs: PROBLEMS, OUTPUT, DEBUG CONSOLE, and TERMINAL. The 'TERMINAL' tab is active, displaying a series of git commands and their outputs in a PowerShell prompt (PS F:\SEM_7\my-git-practice>). The commands and outputs are as follows:

```
PS F:\SEM_7\my-git-practice> git status
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
• PS F:\SEM_7\my-git-practice> git add README.md
• PS F:\SEM_7\my-git-practice> git commit -m "Update README first line on feature branch"
[conflicting-feature 210f37b] Update README first line on feature branch
 1 file changed, 1 insertion(+), 1 deletion(-)
• PS F:\SEM_7\my-git-practice> git checkout main
Switched to branch 'main'
• PS F:\SEM_7\my-git-practice> git merge conflicting-feature
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
• PS F:\SEM_7\my-git-practice> git add README.md
• PS F:\SEM_7\my-git-practice> git status
On branch main
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
  modified:   README.md

• PS F:\SEM_7\my-git-practice> git commit -m "Updated New README"
[main be06e0c] Updated New README
• PS F:\SEM_7\my-git-practice>
```

The bottom status bar shows the current branch as 'main', a 'Run Testcases' button, and a progress indicator showing 0 errors and 0 warnings.